Project Report

on

“TrackIt”

Submitted for the partial fulfilment for the award of the degree of

Bachelor of Technology

In

Computer Science & Engineering

Batch

(2022-2026)

Submitted To:

Er. Bindu Goyal

Submitted by:

Tanveer Kaur

12200672

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DAV UNIVERSITY, JALANDHAR-PATHANKOT NATIONAL HIGHWAY, NH-44, SARMASTPUR PUNJAB, 144012

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Er. Bindu Goyal, for their valuable guidance, constructive feedback, and continuous support throughout the duration of this project. I am also thankful to DR. Rahul Hans and the Department of Bachelor of Technology (Computer Science and Engineering) for providing the necessary resources and an encouraging academic environment.

My appreciation extends to all faculty members and classmates whose suggestions and cooperation helped strengthen the quality of this work. Lastly, I am deeply grateful to my family for their constant encouragement and support, which motivated me to complete this project successfully.

DECLARATION

I, Tanveer Kaur hereby declare that the work which is being presented in this project/training titled “TrackIt” by me, in partial fulfilment of the requirements for the award of Bachelor of Technology (B. Tech) Degree in “Computer Science and Engineering” is an authentic record of my own work carried out under the guidance of Er. Bindu Goyal.

To the best of my knowledge, the matter embodied in this report has not been submitted to any other University/Institute for the award of any degree or diploma.

Tanveer Kaur

12200672

ABSTRACT

In this work, we present the design and implementation of the TrackIt – Lost & Found Campus App. This application is developed using Android with Kotlin and Jetpack Compose, Google's modern UI toolkit that simplifies and enhances the app development experience. TrackIt is intended to solve the common problems faced by students in reporting and locating lost or found items on campus. The traditional method of relying on notice boards or manual communication results in delays and inefficiencies.

The objective of this project is to create a user-friendly mobile application where students can easily upload details of lost or found items, along with images, and receive updates instantly. The app uses Firebase for authentication and database management, Cloudinary for image storage, and OneSignal for push notifications to ensure smooth and real-time communication. TrackIt allows both students and administrators to manage posts effectively, reducing confusion and saving time. In other words, this application provides a centralized digital platform that helps users report, track, and recover lost items within the campus more efficiently and conveniently.

TABLE OF CONTENTS

Sr. No.	TOPIC	PAGE NO.
1.	**INTRODUCTION** 1. Project Overview 2. Objective 3. Key Features 4. Existing System 5. Drawbacks of Existing System 6. Advantages of New System 7. Application	 6 6-7 7-9 9 9-10 10-11 12
2.	**TECHNOLOGY USED** 2.1 Android	12-21
3.	**SRS DOCUMENTATION**	22-25
4.	**FEASIBILITY STUDY** 4.1 Technical Feasibility 4.2 Operational Feasibility 4.3 Economic Feasibility	26-29
5.	**REQUIREMENTS** 5.1 Hardware Requirements 5.2 Software Requirements	30
6.	**DFD** 6.1 0 Level DFD 6.2 1 Level DFD 6.3 2 Level DFD	31-34
7.	**SCREENSHOTS**	35
8.	**FUTURE SCOPE**	36
9.	**CONCLUSION**	37
10.	**BIBLIOGRAPHY**	38

CHAPTER - I

INTRODUCTION

1.1 Project Overview:

In today's world, smartphones have become an essential part of our daily routine due to their ability to simplify tasks and save valuable time. Among all mobile operating systems, Android stands out as the most widely used because of its open-source nature, flexibility, and user-friendly interface. Android is a Linux-based operating system acquired by Google in 2007, and since then, it has powered millions of devices globally, ranging from smartphones to tablets.

The vast availability of Android applications is primarily due to its open-source architecture, which enables developers to build and publish innovative apps with ease. Studies conducted by research organizations such as "Light Castle Partners" indicate that although various operating systems exist, Android remains the most preferred platform among users due to its accessibility and extensive ecosystem.

In this context, the project application has been developed on the Android platform. The name of the application is TrackIt – Lost & Found Campus App. This application is designed to help students and campus members report, search, and recover lost or found items easily through their smartphones. Built using modern Android technologies like Kotlin and Jetpack Compose, TrackIt provides an efficient digital solution to the traditional manual lost-and-found process, ensuring faster communication and improved convenience for users.

1.2Objectives:

Managing lost and found items on a campus often becomes difficult due to the lack of a centralized system, manual reporting, and delays in communication. A lost-and-found management application is designed to streamline the reporting, tracking, and recovery process for misplaced items. The primary objectives of such an application include:

1. **Streamlining Lost & Found Reporting**

- Provide a platform where users can instantly upload details of lost or found items.
- Enable organized listing of items with clear categorization for quick searching.

2. **Ensuring Secure User Identification**
   - Use Firebase Authentication to verify and manage user accounts.
   - Prevent misuse by restricting item uploads and claims to authenticated users only.
3. **Efficient Item Management**
   - Allow users to submit posts with item descriptions and images.
   - Enable editing, updating, and deleting posts as needed.
4. **Admin Monitoring & Verification**
   - Provide administrators with tools to review, approve, or remove posts.
   - Prevent duplicate, false, or inappropriate reports through admin oversight.
5. **Real-Time Notifications**
   - Send push notifications through OneSignal for new matches, claim updates, and admin actions.
   - Ensure users receive timely updates to speed up item recovery.
6. **Enhancing User Experience**
   - Offer an intuitive, easy-to-navigate interface developed with Jetpack Compose.
   - Provide smooth performance, responsive UI, and a modern design for better usability.

These objectives help students and campus administrators save time, reduce confusion, and ensure lost items are handled in a structured, efficient, and professional manner.

1.3 Key Features:

TrackIt – Lost & Found Campus App is designed to streamline and digitalize the entire process of locating, reporting, and managing lost or found items within a campus. The application brings structure, clarity, and speed to what is otherwise a slow and unreliable manual system. Below are the main features of the application, explained in detail:

1. ITEM REPORTING: TrackIt provides a simple and intuitive interface through which users can report lost or found items instantly. Students can upload essential details such as the name of the item, a clear description, the category it belongs to, the approximate location where it was lost or found, and an actual photograph of the item. This ensures that every report is complete, visually identifiable, and helpful for other

users browsing the posts. By digitalizing the reporting process, the app removes the need for notice boards, handwritten forms, or verbal messaging.

2. SECURE USER AUTHENTICATION: To maintain safety and prevent misuse, TrackIt uses Firebase Authentication to register and verify users. Only authenticated students can upload posts, edit their items, or submit claims. This ensures transparency and maintains trust within the app's community. Secure login also helps prevent spam, fake entries, or unauthorized access, making the platform reliable for both students and administrators. User data is stored securely using cloud-based authentication protocols.

3. IMAGE UPLOAD & CLOUD STORAGE: The application allows users to upload photographs of lost or found items for better identification. These images are uploaded and stored securely using Cloudinary, which provides optimized, high-quality image hosting. Fast loading, secure access, and automatic image compression improve performance and clarity. This feature greatly increases the chances of matching items accurately, as images provide an immediate visual reference that plain text descriptions cannot.

4. REAL-TIME NOTIFICATIONS: TrackIt integrates OneSignal push notifications to keep users instantly updated. Whether a new item is reported, someone submits a claim on an item posted by the user, or the admin takes an action such as approving or removing a post, users receive immediate notifications. Real-time communication helps speed up the lost-and-found process and ensures that users never miss important updates. This eliminates delays and encourages quick responses from both users and administrators.

5. ADVANCED SEARCH & FILTER SYSTEM: Users can browse through all reported lost or found items using an easy-to-navigate search interface. The app allows filtering items based on categories, posting dates, item types, or locations, which helps users find relevant posts more quickly. This organized system significantly reduces the time needed to identify items compared to scrolling through notice boards or relying on verbal communication. The search and filtering system creates an effective, user-friendly environment.

6. CLAIM MANAGEMENT SYSTEM: TrackIt includes a complete claim submission and verification system. When a user finds an item that matches someone's lost item, they can submit a claim request with proper details. The owner can review the claim, verify the information, and confirm whether the item belongs to them. This structured claiming process minimizes confusion, prevents disputes, and ensures that items are returned to the rightful owner. Admins also monitor claims to ensure authenticity and avoid false matches.

7. ADMIN PANEL & MODERATION CONTROL: A separate admin panel allows administrators to oversee all app activities. Admins can review new posts, verify their authenticity, approve valid entries, and delete inappropriate or duplicate posts. They also track claims, monitor user activity, and ensure the app remains clean and reliable. This level of control helps maintain the integrity of the system and prevents misuse, providing a disciplined and trustworthy environment for users.

8. MODERN & RESPONSIVE USER INTERFACE: TrackIt is developed using Jetpack Compose, which offers a modern, responsive, and visually appealing UI experience. Composable functions ensure smooth transitions, faster rendering, and an interface that adapts well to different screen sizes. The app combines simplicity with modern design principles, offering clear layouts, organized content, and intuitive navigation. This enhances usability and ensures a comfortable experience for both new and experienced users.

1.4. EXISTING SYSTEM:

- In the existing system, students have to manually visit the campus office, notice board, or administrative department to enquire about lost or found items.
- The process is entirely manual and often time-consuming, leading to delays and miscommunication.
- Lost and found records are usually maintained on paper or through informal verbal communication, making it difficult to track items efficiently.

1.4.1 DRAWBACKS OF EXISTING SYSTEM:

- Handling lost and found records manually is extremely time-consuming and requires continuous effort from staff or students.

● There is a high chance of errors, miscommunication, or loss of information due to the absence of a proper tracking mechanism.

● Students often have to visit multiple places or enquire repeatedly, which creates unnecessary workload and delays the recovery process.

● There is no admin panel or authorized system to verify item reports, manage claims, or ensure the authenticity of information, leading to confusion and sometimes false reporting.

● Paper-based processes and notice-board communication are not secure, as records may get misplaced, damaged, or overlooked easily.

● The manual process lacks transparency and accuracy, making it difficult to track the status of an item or confirm ownership effectively.

1.5. ADVANTAGES OF NEW SYSTEM:

● It saves a significant amount of time by providing a digital platform where students can instantly report lost or found items without visiting any office.

● It helps reduce confusion and prevents common problems such as misplaced records, misinformation, or delays in communication.

● Online data submission is secure because all information is stored safely in Firebase and Cloudinary, reducing the risk of data loss.

● The system provides real-time updates and notifications, allowing users to respond quickly when a matching item is posted.

● It minimizes manual effort and simplifies the overall process, making it faster, more organized, and user-friendly.

● The application is easy to navigate, built using Jetpack Compose, and can be implemented quickly with modern Android technologies.

● Users can easily track the status of their lost items, submitted claims, or uploaded reports anytime through the app.

● Quick and simple registration allows students to create accounts instantly using Firebase Authentication.

● Being mobile-app based, users can access information from anywhere on their smartphones, ensuring convenience and accessibility at all times.

1.6. APPLICATIONS:

● This project can be used in schools to help students and staff report and recover lost items quickly.

● It can be used in hospitals for managing misplaced belongings of patients, visitors, and staff.

● It can be implemented in colleges and universities to streamline lost and found activities across the entire campus.

● It can be used in corporate offices to help employees manage lost items within large workspaces or office buildings.

● It can be used in hostels, PGs, and residential accommodations to track common items that often get misplaced.

● It can be used to notify users instantly when a lost item is found or when an admin verifies or updates a post.

● It can be useful for security departments in institutions who need a centralized system to manage and monitor lost and found inventories.

CHAPTER – II

TECHNOLOGY USED

Kotlin is a modern, trending programming language that was released in 2016 by JetBrains. It has become very popular since it is compatible with Java (one of the most popular programming languages out there), which means that Java code (and libraries) can be used in Kotlin programs. Kotlin is a statically typed programming language developed by JetBrains, the creators of IntelliJ IDEA, and it's now officially supported for Android development by Google.

(Fig1.1)

1. **Conciseness**: Kotlin aims to reduce boilerplate code, which means you can accomplish the same tasks with fewer lines of code compared to Java.

2. **Interoperability**: Kotlin is fully interoperable with Java. This means you can use Kotlin alongside existing Java code, making migration to Kotlin easier for existing Java projects.

3. **Null Safety**: Kotlin's type system is designed to eliminate the risk of null pointer exceptions. Types in Kotlin are non-nullable by default, and you have to explicitly specify when a type can be null.

4. **Extension Functions**: Kotlin allows you to extend existing classes with new functionality without modifying their code. This feature is called extension functions.

5. **Functional Programming Support**: Kotlin has robust support for functional programming concepts like higher-order functions, lambdas, and immutability. This enables you to write more concise and expressive code.
6. **Coroutines**: Kotlin provides first-class support for coroutines, which are a way to perform asynchronous programming. Coroutines make it easier to write asynchronous

code in a sequential style, avoiding the callback hell associated with traditional approaches.

7. **Smart Casts**: Kotlin's type system includes smart casts, which automatically cast types when certain conditions are met. This reduces the need for explicit type casting in your code.

8. **Data Classes**: Kotlin provides a concise way to create classes whose primary purpose is to hold data. These are called data classes, and Kotlin automatically generates **equals()**, **hashCode()**, **toString()**, and **copy()** methods for them.

9. **Companion Objects**: Kotlin supports companion objects, which are similar to static methods and fields in Java. They allow you to define methods and properties that are tied to a class rather than to instances of the class.

10. **Immutable Collections**: Kotlin provides immutable collections by default, which means that once you create a collection, you cannot modify it. This helps prevent accidental modification of data.

Kotlin is used for:

- Mobile applications (especially Android apps)
- Server-side applications
- Data science

Modern Android development focuses on clean architecture, declarative UI, improved performance, and scalable application structure. TrackIt – Lost & Found Campus App is developed using Kotlin, Jetpack Compose, Navigation Component, MVVM Architecture, Hilt Dependency Injection, Coroutines, Firebase, Cloudinary, and OneSignal push notification services.

2.2 Jetpack Compose:

Jetpack Compose is Android's modern UI toolkit that replaces traditional XML layouts.

Compose uses a declarative UI approach, meaning:

- The UI is automatically updated when the app state changes.
- Instead of modifying views manually, developers simply describe what the UI should look like.

2.2.1 Composable Functions:

Everything in Compose begins with a @Composable function:

```kotlin
@Composable
fun MyScreen() {
    Text(text = "Hello Compose")
}
```

A composable function represents a piece of UI. These functions can be reused, combined, and structured to build complex UI layouts.

2.2.2 Recomposition:

Recomposition happens when data/state changes. Compose updates only the required parts of the UI, making it very efficient.

Example:

```kotlin
var count by remember { mutableStateOf(0) }
Button(onClick = { count++ }) {
    Text("Count: $count")
}
```

Whenever count changes, only the Text inside the button recomposes.

2.2.3 State Management in Compose:

Compose uses different ways to handle state:

- mutableStateOf
- ()remember()
- rememberSaveable()
- StateFlow
- LiveData
- ViewModel state hoisting

State hoisting ensures that business logic is separated from the UI, supporting clean architecture.

2.2.4 Modifiers:

Modifiers allow transforming UI elements:

- **Padding**
- **Size**
- **Background**
- **Clickable**
- **Alignment**

Example:

```
Text(
    text = "Hello TrackIt",
    modifier = Modifier
        .padding(16.dp)
        .background(Color.Gray)
)
```

2.2.5 Layouts in Jetpack Compose:

Compose offers layout components similar to XML:

Column – arrange vertically

Row – arrange horizontally

Box – stack components

LazyColumn – scrollable list (replacement for RecyclerView)

LazyRow – horizontal scrolling list

Example:

```
LazyColumn {
    items(itemList) { item ->
        Text(item.title)
    }
}
```

2.2.6 Material Design Components:

Compose supports Material 2 & Material 3 with components like:

- Buttons (Button, FilledButton, OutlinedButton)
- TextFields
- Cards
- Navigation Bars
- FABs (Floating Action Buttons)
- Dialogs
- Snackbars
- Icons
- Scaffolds (structure for screen layout)

Scaffold helps in building a complete screen with top app bar, bottom navigation, and FAB.

2.2.7 Side Effects in Compose:

Compose provides tools to handle logic inside composable functions**:**

- LaunchedEffect – run suspend functions
- SideEffect – run non-suspend actions
- RememberCoroutineScope – launch coroutines inside UI
- DisposableEffect – clean-up tasks

2.2.8 Lifecycle Integration:

Compose works seamlessly with ViewModel and lifecycle owners. It automatically observes changes from:

- LiveData
- Flow
- StateFlow
- SharedFlow

2.3 Navigation Component (Compose Navigation):

Navigation in Jetpack Compose replaces Fragment-based navigation.

2.3.1 NavController

Controls navigation between screens.

2.3.2 NavHost

Defines all the routes in the application.

2.3.3 Passing Arguments

```kotlin
navController.navigate("detailScreen/${item.id}")
```

2.3.4 Backstack Management

Compose automatically maintains the stack of visited screens.

2.3.5 Single-Activity Architecture

The entire TrackIt app uses one activity, and all screens are composable functions.

This leads to:

- Less boilerplate
- Better performance
- Cleaner code structure

2.3 MVVM Architecture (Model–View–ViewModel)

MVVM is the backbone of modern Android architecture.

Model: Handles business logic, Firebase operations, image uploads, and data models.

View: Jetpack Compose screens showing data.

ViewModel: Holds the UI state and exposes:

StateFlow

LiveData

MutableState

This keeps the UI reactive and consistent.

2.5 ViewModel A ViewModel: Survives configuration changes,

Stores UI data,

Manages coroutines using viewModelScope,

Communicates with repositories, Emits states to UI

```kotlin
class LostItemViewModel : ViewModel() {
    val lostItems = MutableStateFlow<List<Item>>(emptyList())
}
```

2.6 Repository Pattern: The repository:

- Fetches data from Firebase Firestore
- Uploads images to Cloudinary
- Handles authentication
- Combines multiple data sources
- Returns clean results to ViewModel
- It ensures that UI does not depend on data sources directly.

```kotlin
class UserRepository @Inject constructor(
    private val auth: FirebaseAuth,
    private val db: FirebaseFirestore
) {

    fun login(email: String, password: String) =
        auth.signInWithEmailAndPassword(email, password)

    fun saveData(data: Any) =
        db.collection("Users").add(data)
}
```

2.7 Hilt Dependency Injection:

Hilt simplifies DI by injecting dependencies automatically.

Key Components:

- @Module – provides dependency functions
- @InstallIn – defines lifecycle scope
- @Singleton – global instance
- @Inject – inject dependencies in classes
- @HiltViewModel – ViewModel injection

Benefits:

- Cleaner code
- Reusability
- Scalable architecture
- Reduces complexity

```
@Module
@InstallIn(SingletonComponent::class)
object AppModule {

    @Provides
    fun provideFirebaseFirestore(): FirebaseFirestore =
        FirebaseFirestore.getInstance()

    @Provides
    fun provideFirebaseAuth(): FirebaseAuth =
        FirebaseAuth.getInstance()
}
```

2.8 Firebase Authentication: Used for secure login and signup.

Features:

- Email/Password authentication
- Token-based user sessions
- Secure access to Firestore data

2.9 Firebase Firestore: Firestore stores all app data:

- Lost items
- Found items
- User profiles
- Claims
- Admin actions

Benefits:

- Real-time updates
- Offline support
- Scalable

- Cloud-based security

2.10 Cloudinary: Used for image uploading and hosting.

Capabilities:

- Auto optimization
- Fast loading URLs
- Secure storage
- Image compression
- Transformations

2.11 OneSignal Push Notifications: OneSignal allows sending push notifications:

- When new items are posted
- When claims are submitted
- When admin verifies posts
- When item matches occur

Notifications improve communication speed and item recovery time.

2.12 Coroutines & Flow: Coroutines manage background tasks:

- Firebase operations
- Database reading
- Image uploads
- Network tasks

Flow & StateFlow:

- Handle real-time data streams
- Update UI automatically
- Work smoothly with Compose

2.3 BASICS OF KOTLIN

Kotlin is a modern, trending programming language. Kotlin is easy to learn, especially if you already know Java (it is 100% compatible with Java). Kotlin is used to develop Android apps, server-side apps, and much more.

1. **Kotlin Syntax -** The fun keyword is used to declare a function. A function is a block of code designed to perform a particular task. In the example above, it declares the main()

function. The main() function is something you will see in every Kotlin program. This function is used to execute code. Any code inside the main() function's curly brackets {} will be executed.

```
fun main(){
println("Hello World")
}
```

2. **Kotlin Variables -** Variables are containers for storing data values .To create a variable, use var or val, and assign a value to it with the equal sign (=):
   a. val: When you create a variable with the val keyword, the value cannot be changed/reassigned.
      val variableName = value
   b. var: When you create a variable with the var keyword, the value can be changed/reassigned.
      var variableName = value

3. **Kotlin Data Types -** In Kotlin, the type of a variable is decided by its value:

```
val myNum=5 // Int 7
val myDoubleNum=5.99 // Double
val myLetter='D' // Char
val myBoolean =true // Boolean
val myText="Hello" // String
```

4. **Kotlin Operator**
   • Arithmetic Operators
   * Multiplication
   / Division
   + Addition
   - Subtraction
   % Remainder/Modulo

CHAPTER - III

SRS Documentation for TrackIt app

3.1. Purpose

Software requirements specification (SRS) is a document that describes what the software will do and how it will be expected to perform. It also describes the functionality the product needs to fulfil all stakeholder needs.

This SRS aims to provide to readers and user information about the system and its functions and specifications. SRS describes the data, functional and behavioural requirements of the software.

3.2 Functional Requirements:

3.2.1 User Panel

- The system shall allow new users to sign up using Name, Registration Number, Email, and Password.
- The system shall allow existing users to login using Registration Number and Password.
- The system shall allow users to reset password via registered email.
- The system shall allow users to create posts for lost/found items with title, description, location, date, and image.
- The system shall upload images to Cloudinary and save the image URL in Firestore.
- The system shall allow users to view their own posts in the "My Posts" screen.
- The system shall allow users to edit or delete their posts.
- The system shall allow users to view all posts in the Home Screen feed.
- The system shall send notifications when post status changes (e.g., approved, resolved).
- The system shall allow users to manage their profile information (name, email, reg no, profile picture).

3.2.2 Admin Panel

- The system shall allow admins to **login** with special credentials.
- The system shall allow admins to **view all posts** created by users.
- The system shall allow admins to **approve, reject, resolve, or delete posts**.
- The system shall maintain a **status field** for each post: Pending, Approved, Resolved.
- The system shall notify users when their posts are **approved, rejected, or resolved**.

3.3 Non-Functional Requirements

3.3.1 Performance Requirements

- The system should support at least **1000 concurrent users**.
- Average **post creation** should take less than **3 seconds** including image upload.
- Notifications should be delivered in **real-time (< 5 seconds delay)**.

3.3.2 Security Requirements

- All user authentication shall be handled by **Firebase Authentication**.
- User passwords shall never be stored in plain text.
- Posts and user data shall be securely stored in **Firebase Firestore**.
- Admin operations shall be **restricted to admin accounts only**.

3.3.3 Usability Requirements

- The UI shall be **simple and intuitive** for non-technical users.
- The app shall be responsive to different screen sizes (phones, tablets).
- Error messages shall be **clear and user-friendly** (e.g., "Invalid Registration Number").

3.3.4 Reliability & Availability

- The system shall have **99% uptime** as it is backed by Firebase services.
- Data shall be **backed up automatically** by Firebase.

3.3.5 Maintainability & Scalability

- The app code shall follow **MVVM architecture** for maintainability.
- The system should allow easy scaling (Firebase auto-scaling).

3.5 User Stories / Use Cases

User Stories (or Use Cases) describe how different types of users interact with the system. These stories help developers understand real-life needs and design the application accordingly. Below are some sample user stories for the TrackIt App, written in simple and clear format:

- **Use Case 1: Upload Lost Item:** As a student, I want to upload the details of a lost item with an image, description, and location. so that other students or staff can help me find it.

- **Use Case 2: Upload Found Item:** As a campus member, I want to post items that I have found so that the rightful owner can identify and claim them easily**.**
- **Use Case 3: Claim an Item:** As a user, I want to submit a claim request for an item posted by someone else so that I can prove the item belongs to me.
- **Use Case 4: Search and Filter Items:** As a student, I want to search and filter lost or found items by category or date so that I can quickly locate relevant posts.
- **Use Case 5: Notification Alerts:** As a user, I want to receive push notifications when new items are uploaded so that I don't miss important updates.
- **Use Case 6: Admin Monitoring:** As an admin, I want to view all posts, claims, and users so that I can monitor misuse, verify claims, and delete false posts.
- **Use Case 7: Login / Registration:** As a new user, I want to register with email and password so that I can securely access the app and post items.

These use cases help describe how the app will be used in real life and support better system planning.

3.6 Constraints:

Constraints define limitations, restrictions, or boundaries under which the system must operate. They ensure the app remains functional, compatible, and realistic to implement.

Here are constraints for your TrackIt app:

1. Platform Constraint: The application must be developed for Android devices only, using Kotlin and Jetpack Compose as per project requirements.

2. Technology Constraint: The system must use:

- Firebase Authentication
- Firestore Database
- Cloudinary for image upload
- OneSignal for notifications

- Only these chosen technologies should be implemented.

3. Network Dependency: Since the app uses Firebase and Cloudinary: A stable internet connection is required. Offline features are limited

4. Budget Constraint: Cloudinary free plan, Firebase free tier, and OneSignal free tier must be used because the project is academic and cost-restricted.

5. Device Compatibility: The app must support:

- Android 8.0 (API Level 26) and above
- Devices with limited storage and RAM
- Different screen sizes using Compose responsive design

6.Security Constraints: The app must ensure**:**

- Only authenticated users can upload posts
- Admin rights cannot be misused
- Data stored must follow Firebase security rules

7.Performance Constraints:

- Image uploads must be optimized
- App should load lists smoothly using LazyColumn
- Database reads should be limited using proper Firestore indexing

8.Storage Constraint: Because Cloudinary free tier is used, image size must be compressed to stay within storage limits.

These constraints guide the developers to build the app within practical limits while ensuring reliability and performance.

CHAPTER – IV

FEASIBILITY STUDY

A feasibility study evaluates all important aspects of a proposed system before it is implemented. It helps determine whether the project can be developed successfully within the available resources and constraints. For the TrackIt – Lost & Found Campus App, the feasibility study includes analysis of operational, technical, economic, market, financial, legal, and scheduling aspects to ensure that the system is practical, achievable, and beneficial.

4.1 OPERATIONAL FEASIBILITY

Operational feasibility examines how well the proposed system will function within the existing environment and how effectively it addresses current problems. The TrackIt application aims to replace the manual lost and found process, which is time-consuming and inefficient.

The system is designed to be simple, user-friendly, and accessible for all campus members. It eliminates manual paperwork by offering digital reporting of lost and found items, real-time notifications, and quick access to data. The app functions smoothly on Android devices with minimum hardware requirements such as:

- 1 GB or above RAM
- Minimum 4 GB storage
- Android 8.0 or higher

TrackIt integrates seamlessly into a campus environment and supports existing procedures by offering features like item posting, claim management, admin verification, and notification services. The system improves maintainability, reliability, and user satisfaction by providing fast and accurate results.

4.2 TECHNICAL FEASIBILITY

Technical feasibility assesses whether the required technology is available and whether the development team has the expertise to build the system. TrackIt uses modern and widely adopted technologies such as Kotlin, Jetpack Compose, Firebase, Cloudinary, and OneSignal, all of which are well-supported.

The app relies on:

- Firebase Authentication for secure login
- Firebase Firestore for cloud data storage
- Cloudinary for image hosting
- OneSignal for push notifications
- Hilt for dependency injection
- MVVM architecture for clean and maintainable code

All these technologies are cloud-based, reliable, and scalable. The development tools required (Android Studio, Firebase console) are readily available, and the skill set needed (Kotlin + Compose) aligns with modern Android development practices. Therefore, the project is technically feasible.

4.3 ECONOMICAL FEASIBILITY

Economic feasibility analyses the cost–benefit ratio of the system. TrackIt is developed using free-tier services such as Firebase, Cloudinary free plan, and OneSignal free usage limits, making the project cost-effective for academic and organizational use.

The benefits of TrackIt include:

- Saving time for students and administrative staff
- Eliminating manual paper records
- Increasing accuracy and reliability of data
- Faster recovery of lost items
- Improved communication through notifications

Since the project uses low or no-cost technologies and provides significant operational benefits, it is economically feasible.

4.4 MARKET ANALYSIS FEASIBILITY

Market feasibility identifies the users and the potential demand for the system. The TrackIt app targets:

- School students
- College and university students
- Campus staff
- Hostel and PG residents

Lost and found problems are common in institutions, making the app relevant and useful. Similar systems exist, but most are web-based or general-purpose. A dedicated campus-level mobile app is more convenient, fast, and user-focused.

A SWOT analysis shows:
Strengths: Fast reporting, user-friendly UI, notification features
Weaknesses: Dependent on internet connectivity
Opportunities: Can be expanded to corporate offices, malls, and public places
Threats: Competition from built-in institutional portals

The market demand for such an app is strong, making the project market feasible.

4.5 FINANCIAL FEASIBILITY

Financial feasibility checks whether the project can be developed and maintained within allowable costs. Since TrackIt uses mostly free platforms:
Development Costs Include:

- App development
- UI/UX design
- Testing
- Deployment
- Maintenance
- Potential Expenses (Minimal):
- Cloudinary upgrades if storage increases
- Firebase billing if high usage

- Play Store developer fees

Since this is a student project and uses mostly free-tier services, the financial investment is low. The app does not rely on revenue models like ads or subscriptions, making the financial feasibility strong.

4.6 LEGAL AND REGULATORY FEASIBILITY

TrackIt must comply with several legal requirements related to handling user data:
Key Legal Considerations:

- User data must be stored securely following privacy standards
- Images and personal information must not be shared with unauthorized users
- Firebase security rules must be implemented
- Users must accept terms and conditions before using the app
- No copyrighted content should be used without permission

Since the app is designed for educational and institutional purposes, legal risks are minimal but manageable. The system is legally feasible when proper guidelines are followed.

4.7 SCHEDULE AND TIME FEASIBILITY

Schedule feasibility determines whether the project can be completed within the allotted timeline. Develop a project timeline outlining key milestones and deliverables. Estimate the time required for each phase of development, including planning, design, development, testing, and deployment. Identify potential risks and dependencies that could impact the project schedule.

CHAPTER – V
REQUIREMENTS

In "TrackIt App" there are basically two requirements: Hardware Requirement & Software Requirement.

Hardware Requirements for PC:

Hardware Requirements for Development (PC/Laptop):

To develop and run the TrackIt application using Android Studio smoothly, the following hardware configuration is recommended:

- Processor: Intel Core i5 or higher (10th Gen recommended)
- RAM: Minimum 8 GB (16 GB recommended for smooth Gradle builds)
- Storage: 512 GB SSD or higher for fast builds and better performance
- Graphics: Integrated or dedicated GPU supporting hardware acceleration
- Operating System: Windows 10/11

These specifications ensure efficient performance during coding, debugging, emulation, and testing in Android Studio.

Software Requirements:

- **Android Studio (Latest Version)**
  Used for developing the TrackIt app using Kotlin and Jetpack Compose.
- **Kotlin Programming Language**
  Official language supported by Google for Android development.
- **Firebase Services**
- **Firebase Authentication**
- **Firebase Firestore**
  Used for secure login and cloud database operations.
- **Cloudinary**
  Used for uploading and storing item images.
- **OneSignal Push Notification Service**
  Used for sending real-time notifications.
- **JDK (Java Development Kit)**

Required for running Android Studio and Gradle builds.

CHAPTER – VI
DATA FLOW DIAGRAM

The Data Flow Diagram (DFD) provides a visual representation of how data moves through the TrackIt – Lost & Found Campus System. It helps in understanding the flow of information between different entities such as users, admin, and the system database. DFD is used as a preliminary step to develop a clear understanding of the application without going into extreme detail at the initial stage.

The diagrams show how data is entered, processed, and stored by the system.

6.1 DFD 0- LEVEL:

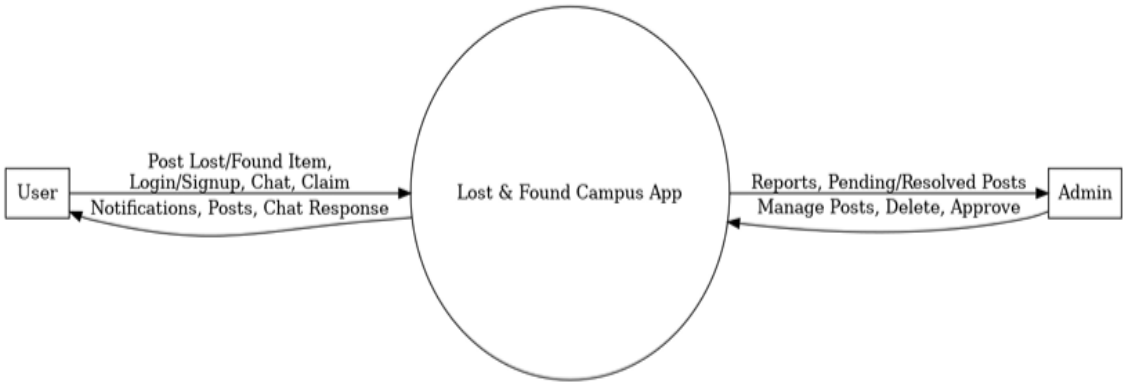


Fig.6.1 DFD 0-level

The 0-Level Data Flow Diagram of the TrackIt – Lost & Found Campus App represents the entire system as a single process and shows how users and admins interact with it. In this level, the user sends data such as lost or found item posts, login or signup information, chat messages, and claim requests into the system. The app then responds back to the user by providing notifications, displaying item posts, and sending chat or claim responses. Similarly, the admin interacts with the system by managing posts, deleting inappropriate entries, and approving claims, while the system provides the admin with reports and lists of pending or resolved posts. This level provides a high-level overview of the overall data flow between the user, the admin, and the TrackIt application without showing internal processes.

6.2 DFD 1-LEVEL :-

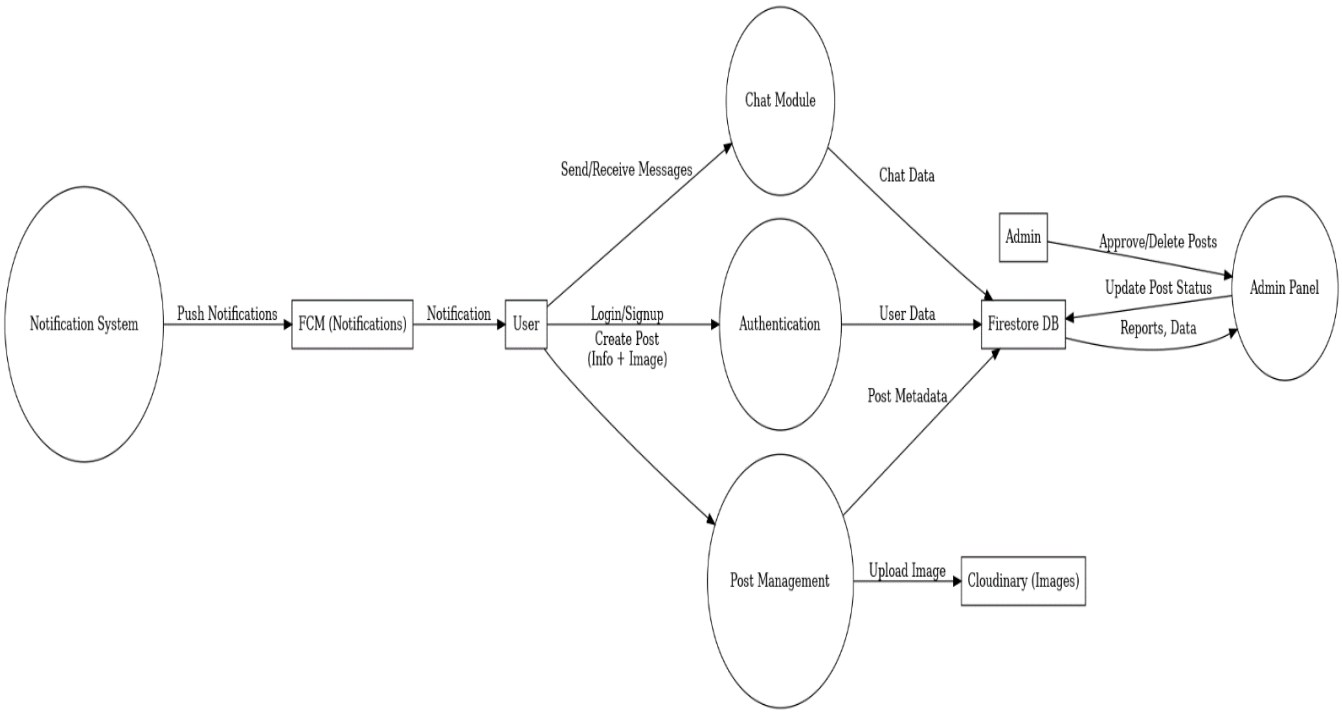


Fig.6.2 DFD 1-Level

The Level-1 Data Flow Diagram of the TrackIt – Lost & Found Campus App provides a more detailed view of how the internal modules of the system interact with each other. In this diagram, the user interacts with multiple components such as the Authentication Module, Chat Module, and Post Management Module. When the user performs login or signup, their information is processed through the Authentication Module and stored in Firestore Database. Similarly, when the user creates a lost or found item post, the Post Management Module handles the item details and uploads the associated image to Cloudinary, while the metadata of the post is stored in Firestore. The Chat Module enables users to send and receive messages related to item claims or verification, and the corresponding chat data is also stored in the database. The Notification System sends push notifications through FCM to alert users about new posts, updates, or admin actions. On the admin side, administrators can approve, delete, or update post statuses through the Admin Panel, and all these actions reflect directly in Firestore Database. Overall, the Level-1 DFD illustrates how key operations such as authentication, posting, image uploading, chatting, and admin management are interconnected inside the TrackIt app.

6.3 DFD 2-LEVEL:-

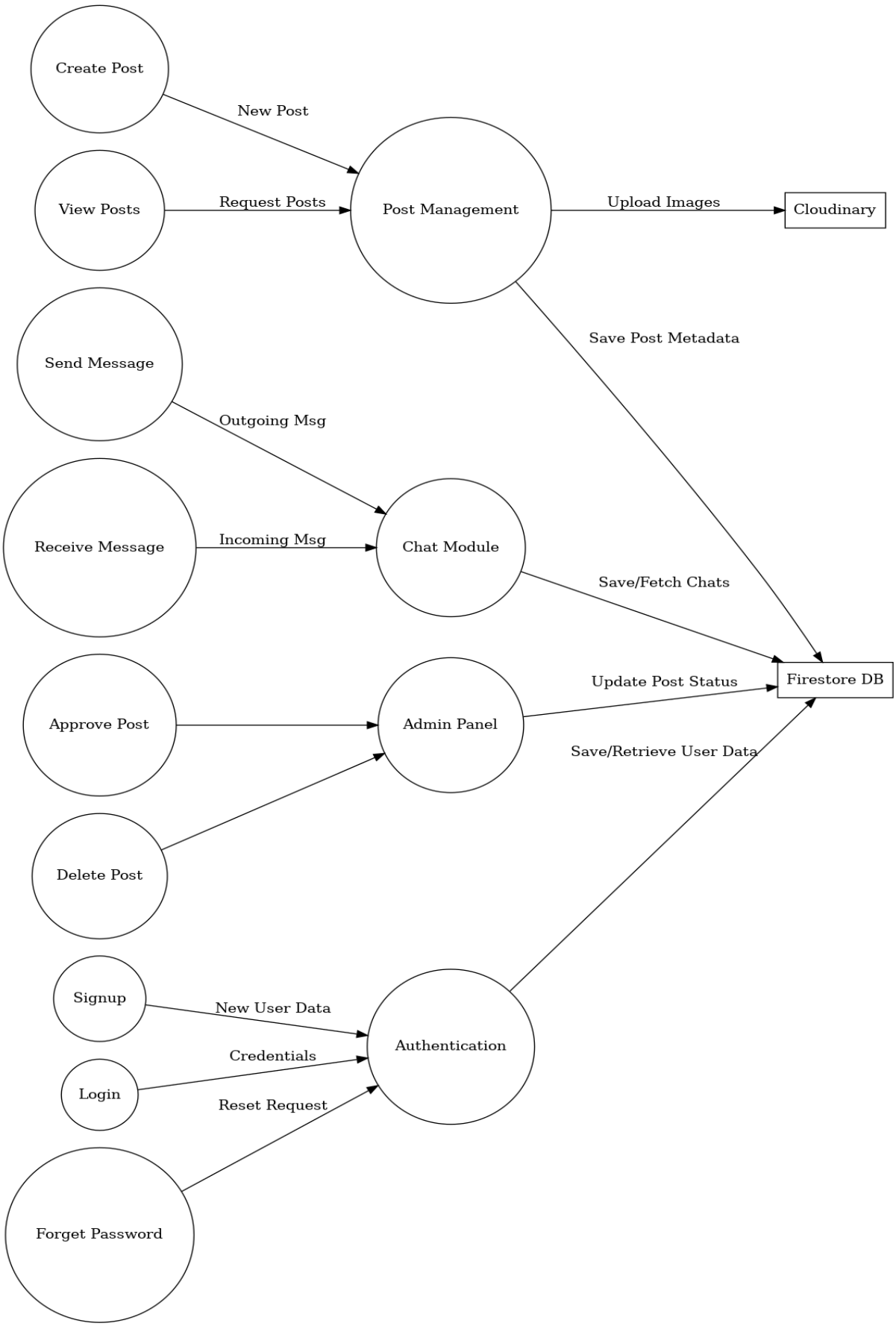
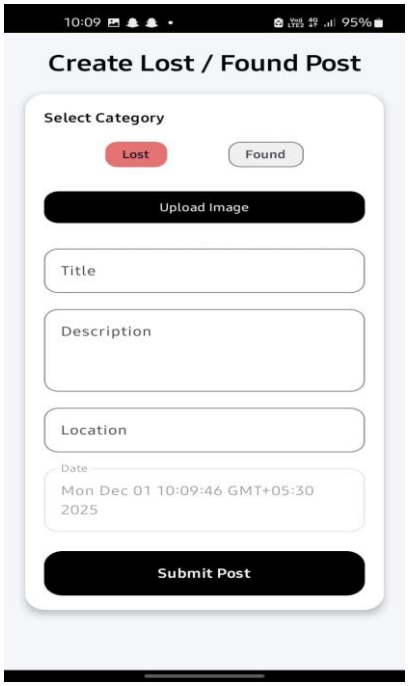


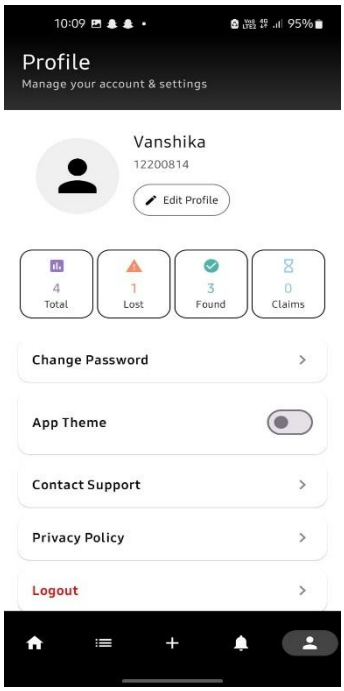
Fig.6.3 DFD 2-LEVEL

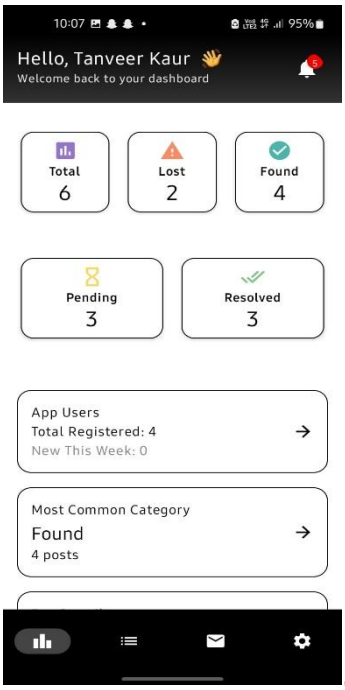
The Level-2 Data Flow Diagram of the TrackIt App explains the detailed functioning of each module inside the system. The Post Management module handles new posts created by users, sends requests to view posts, uploads item images to Cloudinary, and saves post data into Firestore. The Chat Module manages sending and receiving messages between users and stores all chat records in the database. The Admin Panel allows admins to approve or delete posts and update the status of reports directly in Firestore. The Authentication module processes signup data, login credentials, and password reset requests while storing and retrieving user details securely. All modules communicate with Firestore Database to save, update, and fetch necessary data. This level shows a clear breakdown of how posting, chatting, admin actions, and user authentication work together inside the TrackIt system.

CHAPTER VII

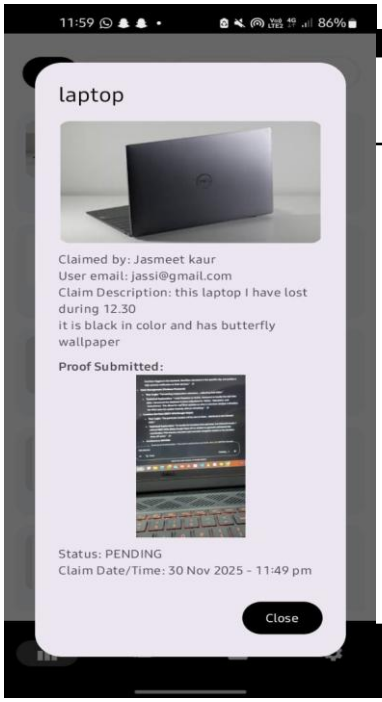
APP SCREENSHOTS

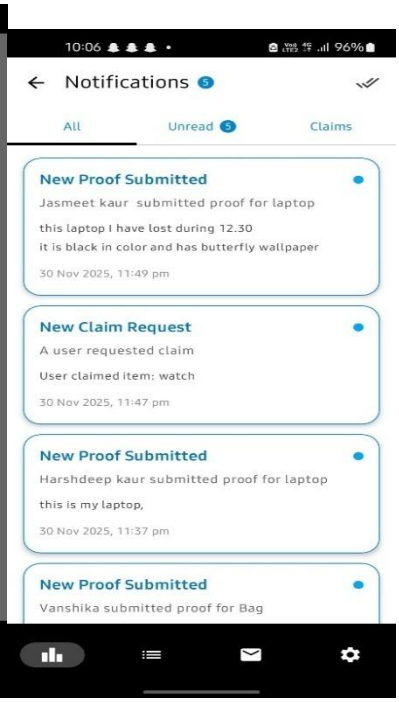


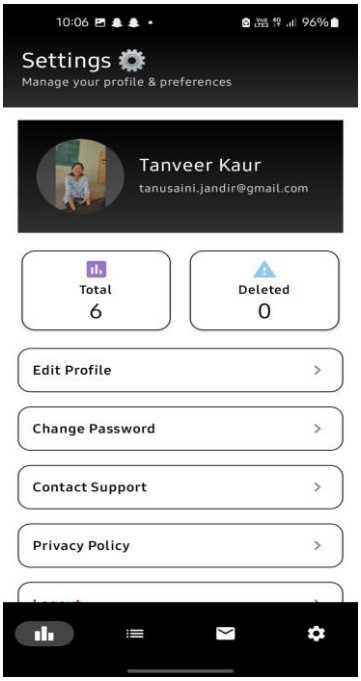




3







CHAPTER VIII

FUTURE SCOPE

The future of the TrackIt app is highly promising as the need for digital lost and found solutions continues to increase across schools, colleges, offices, and public spaces.

● The app can evolve over time by adding more advanced and intelligent features to improve accuracy, user experience, and system efficiency.

● With increasing reliance on smartphones, TrackIt can become a centralized digital platform for managing lost and found items across multiple institutions and campuses.

● Integration of smart technologies such as QR codes, RFID tags, or NFC can help in faster identification and tracking of items.

● Artificial Intelligence (AI) can be used in the future to automatically match lost items with found items based on image recognition and item descriptions.

● Real-time chatbot assistance can be added to help users quickly report items or get updates without navigating through multiple screens.

● A multi-institution version of the app can be developed, allowing different campuses or departments to have their own managed sections under one platform.

● Offline support and local caching can be improved for users with limited internet access.

● A web-based admin dashboard can be created for easier management of posts, users, and claims on larger screens.

● Notifications and communication features can be expanded to support voice messages, attachments, and more personalized alerts.

CHAPTER IX
CONCLUSION

The TrackIt – Lost & Found Campus App is a significant step toward modernizing the traditional lost and found process within educational institutions and other large environments. It provides convenience, improves efficiency, and ensures a smooth, organized experience for students and administrators. By digitalizing item reporting, claim management, and notifications, the app reduces manual effort and minimizes the chances of miscommunication or lost records.

Selecting and using an app like TrackIt requires institutions to understand their needs, user base, and available resources. When chosen and implemented correctly, such a system greatly enhances the ability to manage lost and found activities in a reliable and systematic manner. Overall, TrackIt offers a practical, user-friendly, and scalable solution that simplifies campus operations and contributes to a more connected and responsible community.

CHAPTER X

BIBLIOGRAPHY

- https://developer.android.com/courses/pathways/android-basics-compose-unit-1-pathway-1